

XỬ LÝ, BÁO CÁO GIAO DỊCH THANH TOÁN – FINANCE

Mục tiêu

Xây dựng hai trang: thống kê tài chính và danh sách giao dịch; hỗ trợ tìm kiếm, lọc, phân trang và cập nhật thanh toán COD.

Finance sử dụng hai bảng chính:

- orders: mã đơn, người nhận, số điện thoại, tổng tiền, trạng thái đơn và ngày tạo.
- payment_transactions: order_id, gateway, amount, status, paid_at; lưu các lần thanh toán của đơn hàng.

KIỂM TRA các khai báo đã đầy đủ chưa, nếu chưa bổ sung:

Trong app/Models/Order.php, khai báo quan hệ:

```
public function paymentTransactions()
{
    return $this->hasMany(PaymentTransaction::class);
}
```

Trong app/Models/PaymentTransaction.php, khai báo quan hệ ngược:

```
public function order(): BelongsTo
{
    return $this->belongsTo(Order::class);
}
```

Thực hiện xử lý:

1. Tạo Controller:

2. Cập nhật Controller vừa tạo:

Tổ chức xử lý thành năm phương thức:

- ordersQuery(): nối đơn hàng với một giao dịch đại diện, tránh đếm trùng đơn. Ưu tiên giao dịch đã thu/chờ hoàn/đã hoàn tiền, sau đó chọn ID lớn nhất trong nhóm ưu tiên.
- filteredOrders(): kiểm tra đầu vào; lọc theo mã đơn, tên, điện thoại, khoảng ngày, số tiền, phương thức và trạng thái.
- index(): tính chỉ số và trả về view admin.finance.index.
- transactions(): lấy danh sách, sắp xếp và trả về view admin.finance.transactions.
- updateStatus(): kiểm tra và lưu trạng thái thanh toán COD.

```
<?php

namespace App\Http\Controllers\Admin;

use App\Http\Controllers\Controller;
use App\Models\Order;
use Carbon\Carbon;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use Illuminate\Validation\Rule;
use Illuminate\Validation\ValidationException;

class FinanceController extends Controller
{
    private const STATUSES = [
```

```

    'pending' => 'Chờ thanh toán', 'initiated' => 'Đang chờ MoMo',
    'paid' => 'Đã thanh toán', 'failed' => 'Thanh toán thất bại',
    'cancelled' => 'Đã hủy', 'refund_pending' => 'Chờ hoàn tiền', 'refunded' => 'Đã hoàn tiền',
];

private const COD_TRANSITIONS = [
    'pending' => ['pending', 'paid', 'failed'],
    'failed' => ['failed', 'pending', 'paid'],
    'paid' => ['paid', 'refund_pending'],
    'refund_pending' => ['refund_pending', 'refunded'],
    'refunded' => ['refunded'], 'cancelled' => ['cancelled'],
];

// Ưu tiên giao dịch đã thu/hoàn tiền; lần thử thanh toán mới không che mất tiền đã thu.
private const PAYMENT_PRIORITY = "CASE WHEN status IN ('paid', 'refund_pending', 'refunded') THEN 0 ELSE 1 END";

private function ordersQuery()
{
    $paymentId = DB::table('payment_transactions')->select('id')
        ->whereColumn('order_id', 'orders.id')->orderByRaw(self::PAYMENT_PRIORITY)->orderByDesc('id')->limit(1);

    $orders = DB::table('orders')->leftJoin('payment_transactions as payment', function ($join) use ($paymentId) {
        $join->on('payment.order_id', '=', 'orders.id')->where('payment.id', '=', $paymentId);
    })->select('orders.*', 'payment.id as payment_id', 'payment.paid_at')
        ->selectRaw("COALESCE(payment.gateway, CASE WHEN orders.status IN ('cod_ordered', 'cod_paid') THEN 'cod' WHEN
orders.status IN ('paid', 'paid_momo') THEN 'momo' ELSE 'unknown' END) as gateway")
        ->selectRaw("COALESCE(payment.status, CASE WHEN orders.status = 'cod_ordered' THEN 'pending' WHEN orders.status IN
('cod_paid', 'paid_momo') THEN 'paid' ELSE orders.status END) as payment_status");

    return DB::query()->fromSub($orders, 'finance_orders');
}

private function filteredOrders(Request $request): array
{

```

```

$filters = $request->validate([
    'search' => ['nullable', 'string', 'max:100'],
    'date_from' => ['nullable', 'date_format:Y-m-d'],
    'date_to' => ['nullable', 'date_format:Y-m-d', ...($request->filled('date_from') ? ['after_or_equal:date_from'] :
[])],
    'min_amount' => ['nullable', 'numeric', 'min:0', 'max:999999999999.99'],
    'max_amount' => ['nullable', 'numeric', 'min:0', 'max:999999999999.99', ...($request->filled('min_amount') ?
['gte:min_amount'] : [])],
    'gateway' => ['nullable', Rule::in(['cod', 'momo', 'unknown'])],
    'payment_status' => ['nullable', Rule::in(array_keys(self::STATUSES))],
    'sort' => ['nullable', Rule::in(['newest', 'oldest', 'amount_asc', 'amount_desc'])],
    'page' => ['nullable', 'integer', 'min:1'],
], [
    'date_to.after_or_equal' => 'Ngày kết thúc phải từ ngày bắt đầu trở đi.',
    'max_amount.gte' => 'Số tiền tối đa phải lớn hơn hoặc bằng số tiền tối thiểu.',
    '*.date_format' => 'Ngày lọc không hợp lệ (định dạng năm-tháng-ngày).',
    '*.numeric' => 'Số tiền phải là một giá trị số.',
    '*.min' => 'Giá trị bộ lọc nhỏ hơn mức cho phép.',
    '*.in' => 'Giá trị bộ lọc không hợp lệ.',
]);

$query = $this->ordersQuery()->where('created_at', '<=', now());
if ($request->filled('search')) {
    $search = trim($filters['search']);
    $query->where(function ($query) use ($search) {
        $query->where('name', 'like', '%'.$search.'%')->orWhere('phone', 'like', '%'.$search.'%');
        if (ctype_digit(ltrim($search, '#'))) {
            $query->orWhere('id', ltrim($search, '#'));
        }
    });
}
foreach (['gateway', 'payment_status'] as $field) {
    if ($request->filled($field)) {
        $query->where($field, $filters[$field]);
    }
}

```

```

    }
}
if ($request->filled('date_from')) {
    $query->where('created_at', '>=', Carbon::parse($filters['date_from'])->startOfDay());
}
if ($request->filled('date_to')) {
    $query->where('created_at', '<', Carbon::parse($filters['date_to'])->addDay()->startOfDay());
}
foreach ([ 'min_amount' => '>=', 'max_amount' => '<=' ] as $field => $operator) {
    if ($request->filled($field)) {
        $query->where('total_price', $operator, $filters[$field]);
    }
}

return [$query, $filters];
}

public function index(Request $request)
{
    [$query, $filters] = $this->filteredOrders($request);

    // Thống kê toàn bộ kết quả lọc; mỗi đơn chỉ tính một lần.
    $summary = (clone $query)->selectRaw('COUNT(*) as order_count, COALESCE(SUM(total_price), 0) as total_amount')->first();
    $statusTotals = (clone $query)->select('payment_status')
        ->selectRaw('COUNT(*) as order_count, SUM(total_price) as total_amount')
        ->groupBy('payment_status')->get()->keyBy('payment_status');
    $methodTotals = (clone $query)->select('gateway')
        ->selectRaw('COUNT(*) as order_count, SUM(total_price) as total_amount')
        ->selectRaw("SUM(CASE WHEN payment_status = 'paid' THEN total_price ELSE 0 END) as paid_amount")
        ->groupBy('gateway')->get()->keyBy('gateway');

    return view('admin.finance.index', [
        'filters' => $filters, 'summary' => $summary,
        'statusTotals' => $statusTotals, 'methodTotals' => $methodTotals,
    ]);
}

```

```

        'statuses' => self::STATUSES,
        'methods' => ['cod' => 'COD', 'momo' => 'MoMo', 'unknown' => 'Chưa xác định'],
    ]);
}

public function transactions(Request $request)
{
    [$query, $filters] = $this->filteredOrders($request);

    [$column, $direction] = match ($filters['sort'] ?? 'newest') {
        'oldest' => ['created_at', 'asc'], 'amount_asc' => ['total_price', 'asc'],
        'amount_desc' => ['total_price', 'desc'], default => ['created_at', 'desc'],
    };
    $orders = $query->orderBy($column, $direction)->orderBy('id', $direction)->paginate(15)->withQueryString();

    return view('admin.finance.transactions', [
        'orders' => $orders, 'filters' => $filters,
        'statuses' => self::STATUSES, 'codTransitions' => self::COD_TRANSITIONS,
        'methods' => ['cod' => 'COD', 'momo' => 'MoMo', 'unknown' => 'Chưa xác định'],
    ]);
}

public function updateStatus(Request $request, Order $order)
{
    $data = $request->validate([
        'payment_status' => ['required', Rule::in(array_keys(self::COD_TRANSITIONS))],
        'current_payment_status' => ['required', 'string'],
        'current_order_status' => ['required', 'string'],
        'current_payment_id' => ['required', 'integer', 'min:0'],
    ], ['payment_status.in' => 'Trạng thái COD không hợp lệ.', '*.required' => 'Thiếu thông tin trạng thái. Vui lòng tải lại trang.']);

    DB::transaction(function () use ($order, $data, $request) {
        $order = Order::whereKey($order->id)->lockForUpdate()->firstOrFail();
    });
}

```

```

        $payment = $order->paymentTransactions()->orderByRaw(self::PAYMENT_PRIORITY)->orderByDesc('id')->lockForUpdate()-
>first();
        $isCod = $payment ? $payment->gateway === 'cod' : in_array($order->status, ['cod_ordered', 'cod_paid'], true);
        if (!$isCod) {
            throw ValidationException::withMessages(['payment_status' => 'Chỉ được cập nhật thủ công cho đơn COD.']);
        }
        $currentStatus = $payment->status ?? ($order->status === 'cod_paid' ? 'paid' : 'pending');
        if ($currentStatus !== $data['current_payment_status'] || $order->status !== $data['current_order_status'] || (int)
($payment->id ?? 0) !== (int) $data['current_payment_id']) {
            throw ValidationException::withMessages(['payment_status' => 'Đơn hàng vừa thay đổi. Vui lòng tải lại trang trước
khi cập nhật.']);
        }
        $newStatus = $data['payment_status'];
        if (!in_array($newStatus, self::COD_TRANSITIONS[$currentStatus] ?? [], true)) {
            throw ValidationException::withMessages(['payment_status' => 'Không thể chuyển sang trạng thái thanh toán
này.']);
        }
        if ($newStatus === $currentStatus) {
            return;
        }
        if (in_array($newStatus, ['pending', 'paid'], true) && ($order->status === 'cancelled' || in_array($order-
>shipping_status, ['cancelled', 'return', 'returned'], true))) {
            throw ValidationException::withMessages(['payment_status' => 'Không thể xác nhận thu tiền cho đơn đã hủy hoặc
hoàn hàng.']);
        }
        $attributes = [
            'status' => $newStatus,
            'message' => 'Quản trị viên #' . $request->user()->id . ' cập nhật: ' . self::STATUSES[$newStatus],
            'paid_at' => $newStatus === 'paid' ? ($payment->paid_at ?? now()) : $payment->paid_at,
        ];
        if ($payment) {
            $payment->update($attributes);
        } else {

```

```

        $order->paymentTransactions()->create(array_merge($attributes, ['gateway' => 'cod', 'amount' => $order->total_price]));
    }
    if ($newStatus === 'paid') {
        $order->update(['status' => 'cod_paid']);
    } elseif (in_array($newStatus, ['pending', 'failed'], true)) {
        $order->update(['status' => 'cod_ordered']);
    }
}));

return back()->with('success', 'Đã lưu trạng thái thanh toán đơn COD #'.$order->id.'.');
}
}

```

3. Khai báo Route và quyền admin

Trong routes/web.php, thêm import và các route dưới đây vào nhóm admin hiện có:

```

use App\Http\Controllers\Admin\FinanceController;

Route::middleware(['auth', 'admin'])->prefix('admin')->name('admin.')->group(function () {
    Route::get('/finance', [FinanceController::class, 'index'])->name('finance.index');
    Route::get('/finance/transactions', [FinanceController::class, 'transactions'])->name('finance.transactions');
    Route::patch('/finance/{order}/status', [FinanceController::class, 'updateStatus'])->name('finance.update-status');
});

```

4. Xây dựng hai trang giao diện

Trang 1: hiển thị chỉ số thống kê, bảng biểu báo cáo, xuất dữ liệu

SHOP ADMIN

S

Shop Admin

QUẢN LÝ BÁN HÀNG

Dashboard

Sản phẩm

Danh mục

Người dùng

Đơn hàng

Thống kê tài chính

Giao dịch thanh toán

Báo cáo

Đăng xuất

Quản trị / Thống kê tài chính

Shop Admin

Quản trị viên

Thống kê tài chính

Tổng hợp giá trị thanh toán theo trạng thái và phương thức.

Thống kê chỉ số

Giao dịch thanh toán

Tìm đơn hàng

Mã đơn, tên hoặc số điện thoại

Từ ngày tạo đơn

dd/mm/yyyy

Đến ngày tạo đơn

dd/mm/yyyy

Số tiền từ (đ)

Không giới hạn

Số tiền đến (đ)

Không giới hạn

Phương thức

Tất cả

Trạng thái thanh toán

Tất cả

Áp dụng bộ lọc

Xóa bộ lọc

Có 0 đơn phù hợp. Số tiền bao gồm phí vận chuyển, thống kê theo ngày tạo đơn trên toàn bộ kết quả lọc.

Tổng giá trị đơn hàng

0 đ

0 đơn, bao gồm đơn đã hủy

Chờ thanh toán

0 đ

0 đơn

Đang chờ MoMo

0 đ

0 đơn

Đã thanh toán

0 đ

0 đơn

Thanh toán thất bại

0 đ

0 đơn

Đã hủy

0 đ

0 đơn

Chờ hoàn tiền

0 đ

0 đơn

Đã hoàn tiền

0 đ

0 đơn

Thống kê theo phương thức

Phương thức	Số đơn	Tổng giá trị	Đã thanh toán
COD	0	0 đ	0 đ
MoMo	0	0 đ	0 đ
Chưa xác định	0	0 đ	0 đ

Chat Khách hàng

Trang 2: Hiện thị danh sách chi tiết các giá trị thanh toán và cho phép cập nhật các giá trị các đơn hàng COD

SHOP ADMIN

S

Shop Admin

QUẢN LÝ BÁN HÀNG

Dashboard

Sản phẩm

Danh mục

Người dùng

Đơn hàng

Thống kê tài chính

Giao dịch thanh toán

Báo cáo

Đăng xuất

Quản trị / Giao dịch thanh toán

Shop AdminQuản trị viên

Giao dịch thanh toán

Thống kê chi số

Giao dịch thanh toán

Tìm đơn hàng

Mã đơn, tên hoặc số điện thoại

Từ ngày tạo đơn

dd/mm/yyyy

Đến ngày tạo đơn

dd/mm/yyyy

Số tiền từ (đ)

Không giới hạn

Số tiền đến (đ)

Không giới hạn

Phương thức

Tất cả

Trạng thái thanh toán

Tất cả

Sắp xếp

Mới nhất

Áp dụng bộ lọc

Xóa bộ lọc

Có 0 đơn phù hợp bộ lọc. Số tiền bao gồm phí vận chuyển; ngày lọc là ngày tạo đơn.

Danh sách giao dịch (0 đơn)

COD: xác nhận thu tiền hoặc thất bại; đơn đã thu tiền có thể chuyển sang chờ hoàn tiền rồi xác nhận đã hoàn tiền.

Đơn hàng	Khách hàng	Phương thức	Số tiền	Thanh toán	Cập nhật COD
Không có đơn hàng phù hợp với bộ lọc.					

Chat Khách hàng